

Definições dos Padrões de Projeto

Objetivo do arquivo

Resumo objetivo dos padrões de projeto estudados na disciplina, com foco no problema que cada um resolve, na definição, nas vantagens e nas desvantagens.

A ideia é servir como revisão rápida para prova, usando a terminologia de projeto associada a acoplamento, coesão, encapsulamento, reuso e extensibilidade.

Visão geral

Categoria	Padrão	Ideia central
Criacional	Factory Method	Delega para subclasses a decisão de qual objeto criar
Criacional	Abstract Factory	Cria famílias de objetos relacionados sem expor classes concretas
Criacional	Builder	Separa a construção de um objeto complexo da sua representação
Criacional	Prototype	Cria novos objetos por cópia de um objeto existente
Criacional	Singleton	Garante uma única instância e um ponto global de acesso
Estrutural	Adapter	Converte uma interface em outra esperada pelo cliente
Estrutural	Bridge	Separa abstração e implementação para variar as duas de forma independente
Estrutural	Composite	Agrupa objetos em estruturas de árvore e trata parte e todo de forma uniforme
Estrutural	Decorator	Adiciona responsabilidades dinamicamente por composição
Estrutural	Facade	Oferece uma interface simples para um subsistema complexo
Estrutural	Flyweight	Compartilha estado para reduzir uso de memória e objetos repetidos
Estrutural	Proxy	Controla o acesso a outro objeto por meio de um representante
Comportamental	Chain of Responsibility	Encadeia objetos até que um deles trate a requisição
Comportamental	Command	Encapsula uma requisição como objeto
Comportamental	Interpreter	Representa uma gramática e interpreta sentenças dela

Categoria	Padrão	Ideia central
Comportamental	Iterator	Acessa elementos de uma coleção sem expor sua estrutura interna
Comportamental	Mediator	Centraliza a comunicação entre objetos relacionados
Comportamental	Memento	Captura e restaura estado sem expor sua estrutura interna
Comportamental	Observer	Notifica vários objetos quando um estado muda
Comportamental	State	Altera o comportamento conforme o estado interno
Comportamental	Strategy	Troca algoritmos ou comportamentos de forma intercambiável
Comportamental	Template Method	Define o esqueleto de um algoritmo e deixa passos para subclasses
Comportamental	Visitor	Separa operações da estrutura de objetos visitados

Padrões Criacionais

Factory Method

- Categoria: Criacional
- Problema que soluciona: evita acoplamento direto do cliente com classes concretas e centraliza a decisão de criação.
- Definição: define um método responsável por criar objetos, deixando subclasses decidir qual classe concreta será instanciada.
- Como funciona: a classe base ou fábrica define o método; subclasses ou variações concretas retornam o objeto adequado.
- Vantagens:
 - reduz acoplamento;
 - encapsula criação;
 - favorece extensão sem modificar código existente;
 - facilita manutenção;
 - evita espalhar **new** pelo sistema.
- Desvantagens:
 - aumenta número de classes;
 - pode introduzir indireção;
 - pode ser exagero para criação simples.
- Relação com princípios de projeto: ajuda no Aberto/Fechado e na Inversão de Dependência; também favorece programar para interfaces, não para implementações.
- Quando costuma aparecer: quando a criação varia conforme tipo, configuração ou contexto.

Abstract Factory

- Categoria: Criacional
- Problema que soluciona: cria famílias de objetos compatíveis sem prender o cliente às classes concretas.

- Definição: fornece uma interface para criar conjuntos de objetos relacionados ou dependentes.
- Como funciona: uma fábrica abstrata define métodos de criação; fábricas concretas retornam produtos da mesma família.
- Vantagens:
 - reduz acoplamento;
 - encapsula criação de famílias;
 - melhora consistência entre objetos relacionados;
 - facilita troca de plataforma, tema ou variante;
 - favorece reuso e extensão.
- Desvantagens:
 - aumenta a estrutura do sistema;
 - pode gerar muitas classes;
 - é mais difícil de entender no início.
- Relação com princípios de projeto: ajuda em Inversão de Dependência, Aberto/Fechado e baixo acoplamento.
- Quando costuma aparecer: quando o sistema precisa trocar uma família completa de objetos, como interfaces de uma mesma plataforma.

Builder

- Categoria: Criacional
- Problema que soluciona: construção complexa com muitos passos ou opções.
- Definição: separa a montagem de um objeto da sua representação final.
- Como funciona: um builder executa passos de construção e, no final, entrega o objeto pronto.
- Vantagens:
 - separa responsabilidades;
 - melhora legibilidade;
 - facilita criação de objetos complexos;
 - evita construtores gigantes;
 - favorece reuso da sequência de montagem.
- Desvantagens:
 - adiciona classes e etapas;
 - pode ser excesso para objetos simples;
 - pode aumentar indireção.
- Relação com princípios de projeto: melhora coesão e separação de responsabilidades; pode apoiar Aberto/Fechado quando a construção varia.
- Quando costuma aparecer: quando um objeto tem muitos atributos opcionais ou montagem em etapas.

Prototype

- Categoria: Criacional
- Problema que soluciona: criação repetida de objetos semelhantes com custo alto de montagem.
- Definição: cria novos objetos copiando um protótipo existente.
- Como funciona: o cliente solicita uma cópia de um objeto-base já configurado.
- Vantagens:
 - evita repetição de configuração;

- melhora desempenho em certos cenários;
- facilita reuso de estado inicial;
- reduz complexidade de criação.
- Desvantagens:
 - pode ser difícil definir cópia profunda ou rasa;
 - pode gerar problemas com referências internas;
 - aumenta cuidado com clonagem.
- Relação com princípios de projeto: favorece reuso e pode reduzir acoplamento com a lógica de criação.
- Quando costuma aparecer: quando criar um objeto do zero é caro ou quando já existe um modelo pronto.

Singleton

- Categoria: Criacional
- Problema que soluciona: necessidade de uma única instância compartilhada.
- Definição: garante que uma classe tenha no máximo uma instância e fornece acesso global a ela.
- Como funciona: a classe controla sua própria criação e retorna sempre a mesma instância.
- Vantagens:
 - centraliza acesso;
 - pode simplificar recursos únicos;
 - evita múltiplas instâncias quando isso não é desejado.
- Desvantagens:
 - pode criar acoplamento global;
 - esconde dependências;
 - dificulta testes;
 - pode virar overengineering.
- Relação com princípios de projeto: pode prejudicar Inversão de Dependência e baixo acoplamento; exige cuidado com manutenibilidade.
- Quando costuma aparecer: quando existe um recurso realmente único, como configuração global ou serviço compartilhado.

Padrões Estruturais

Adapter

- Categoria: Estrutural
- Problema que soluciona: incompatibilidade entre interfaces.
- Definição: converte a interface de uma classe em outra esperada pelo cliente.
- Como funciona: o adaptador recebe o objeto existente e expõe a interface desejada, delegando chamadas internamente.
- Vantagens:
 - reduz acoplamento com classes concretas externas;
 - facilita integração de componentes incompatíveis;
 - encapsula adaptação;
 - favorece reuso.
- Desvantagens:

- adiciona camada extra;
- pode complicar entendimento;
- depende da interface alvo correta.
- Relação com princípios de projeto: ajuda Inversão de Dependência, favorece Aberto/Fechado e programar para interfaces.
- Quando costuma aparecer: quando uma API ou classe existente não bate com o contrato esperado pelo cliente.

Bridge

- Categoria: Estrutural
- Problema que soluciona: explosão de subclasses quando abstração e implementação variam de forma independente.
- Definição: separa a abstração da implementação em duas hierarquias conectadas por composição.
- Como funciona: a abstração guarda uma referência para a implementação e delega o trabalho para ela.
- Vantagens:
 - reduz acoplamento entre abstração e implementação;
 - facilita extensão independente;
 - evita explosão combinatória de classes;
 - favorece reuso.
- Desvantagens:
 - aumenta indireção;
 - adiciona mais classes e interfaces;
 - pode ser desnecessário para casos simples.
- Relação com princípios de projeto: reforça baixo acoplamento, Inversão de Dependência e Aberto/Fechado.
- Quando costuma aparecer: quando há duas dimensões de variação que não devem crescer juntas.

Composite

- Categoria: Estrutural
- Problema que soluciona: tratamento uniforme de objetos individuais e composições de objetos.
- Definição: organiza objetos em estrutura de árvore para representar parte e todo.
- Como funciona: componentes e compostos implementam a mesma interface; o cliente trata ambos de forma parecida.
- Vantagens:
 - facilita reuso;
 - simplifica o cliente;
 - trata hierarquias de forma uniforme;
 - separa responsabilidades.
- Desvantagens:
 - pode dificultar controle fino de tipos;
 - pode esconder diferenças entre folha e composto;
 - estrutura pode ficar mais complexa.
- Relação com princípios de projeto: favorece composição, reuso e baixo acoplamento com a estrutura interna.

- Quando costuma aparecer: em árvores de objetos, listas de componentes, menus e estruturas hierárquicas.

Decorator

- Categoria: Estrutural
- Problema que soluciona: adição de responsabilidades sem criar muitas subclasses.
- Definição: adiciona comportamento a um objeto dinamicamente por composição.
- Como funciona: um decorator implementa a mesma interface do objeto base e envolve outro objeto da mesma interface.
- Vantagens:
 - encapsula variação;
 - reduz necessidade de herança;
 - favorece extensão sem modificar código existente;
 - separa responsabilidades;
 - melhora reuso de comportamentos combináveis.
- Desvantagens:
 - pode gerar muitas camadas;
 - pode dificultar depuração;
 - aumenta complexidade estrutural.
- Relação com princípios de projeto: ajuda Aberto/Fechado, coesão e baixo acoplamento.
- Quando costuma aparecer: quando comportamentos precisam ser combinados em tempo de execução.

Facade

- Categoria: Estrutural
- Problema que soluciona: interface complexa de subsistema.
- Definição: fornece uma interface simples e unificada para um conjunto de classes mais complexas.
- Como funciona: a fachada coordena chamadas internas e expõe operações de alto nível ao cliente.
- Vantagens:
 - reduz acoplamento;
 - simplifica uso;
 - separa responsabilidades;
 - melhora legibilidade;
 - esconde detalhes internos.
- Desvantagens:
 - pode virar classe grande demais;
 - pode concentrar muita responsabilidade;
 - pode esconder excesso de complexidade do subsistema.
- Relação com princípios de projeto: ajuda Responsabilidade Única se bem usada, e favorece baixo acoplamento e encapsulamento.
- Quando costuma aparecer: quando o cliente não deve interagir com várias classes do subsistema diretamente.

Flyweight

- Categoria: Estrutural

- Problema que soluciona: consumo excessivo de memória por muitos objetos semelhantes.
- Definição: compartilha estado entre objetos para reduzir uso de recursos.
- Como funciona: divide o estado em parte compartilhada e parte externa; o estado compartilhado é reutilizado.
- Vantagens:
 - reduz memória;
 - favorece reuso;
 - evita duplicação de dados iguais;
 - melhora eficiência em grande escala.
- Desvantagens:
 - aumenta complexidade;
 - exige separar estado interno e externo;
 - pode dificultar entendimento inicial.
- Relação com princípios de projeto: favorece reuso e pode melhorar manutenibilidade em cenários com muitos objetos repetidos.
- Quando costuma aparecer: quando há muitos objetos muito parecidos, como caracteres, tokens ou elementos gráficos.

Proxy

- Categoria: Estrutural
- Problema que soluciona: controle de acesso a um objeto real.
- Definição: fornece um representante que intercepta ou controla as chamadas antes de repassá-las ao objeto real.
- Como funciona: o cliente fala com o proxy, e o proxy decide quando e como acessar o objeto real.
- Vantagens:
 - reduz acoplamento;
 - encapsula acesso;
 - permite controle de cache, log, segurança ou carregamento tardio;
 - separa responsabilidades.
- Desvantagens:
 - adiciona indireção;
 - pode complicar depuração;
 - pode ser exagero se não houver regra extra de acesso.
- Relação com princípios de projeto: ajuda Inversão de Dependência, Aberto/Fechado e baixo acoplamento.
- Quando costuma aparecer: quando há necessidade de controlar acesso, adiar criação, cachear ou proteger um objeto.

Padrões Comportamentais

Chain of Responsibility

- Categoria: Comportamental
- Problema que soluciona: decidir quem trata uma requisição sem acoplar o cliente a um handler específico.
- Definição: passa a requisição por uma cadeia de objetos até que algum deles a trate.

- Como funciona: cada objeto decide tratar a requisição ou encaminhá-la para o próximo.
- Vantagens:
 - reduz acoplamento;
 - separa responsabilidades;
 - facilita extensão da cadeia;
 - evita dependência de um único handler.
- Desvantagens:
 - pode ficar difícil rastrear o fluxo;
 - pode gerar indireção excessiva;
 - a ordem da cadeia importa.
- Relação com princípios de projeto: favorece baixo acoplamento, coesão e extensão sem alterar o cliente.
- Quando costuma aparecer: quando várias regras podem processar uma mesma solicitação.

Command

- Categoria: Comportamental
- Problema que soluciona: desacoplar quem pede uma ação de quem a executa.
- Definição: encapsula uma requisição como objeto.
- Como funciona: o comando guarda os dados necessários e expõe uma operação padrão para execução.
- Vantagens:
 - separa responsabilidades;
 - facilita filas, logs e desfazer/refazer;
 - reduz acoplamento;
 - favorece reuso de ações.
- Desvantagens:
 - adiciona muitas classes;
 - pode parecer indireto para ações simples;
 - pode aumentar complexidade estrutural.
- Relação com princípios de projeto: ajuda Inversão de Dependência, encapsulamento e baixo acoplamento.
- Quando costuma aparecer: quando a ação precisa ser tratada como objeto, fila ou histórico.

Interpreter

- Categoria: Comportamental
- Problema que soluciona: interpretar expressões definidas por uma gramática simples.
- Definição: representa a gramática de uma linguagem e interpreta sentenças dessa gramática.
- Como funciona: cada regra da gramática vira uma classe ou estrutura que colabora na interpretação.
- Vantagens:
 - separa regras de interpretação;
 - facilita extensão da gramática em casos simples;
 - organiza a lógica de linguagem.
- Desvantagens:
 - cresce rápido;
 - pode ficar verboso;

- não é adequado para gramáticas complexas.
- Relação com princípios de projeto: favorece separação de responsabilidades, mas pode aumentar complexidade e acoplamento interno.
- Quando costuma aparecer: quando o sistema precisa entender expressões ou linguagens pequenas.

Iterator

- Categoria: Comportamental
- Problema que soluciona: percorrer coleções sem expor sua estrutura interna.
- Definição: fornece uma forma padronizada de acessar elementos sequencialmente.
- Como funciona: o iterator encapsula a posição atual e avança até o fim da coleção.
- Vantagens:
 - esconde detalhes da coleção;
 - separa responsabilidades;
 - facilita reuso do mecanismo de travessia;
 - reduz acoplamento.
- Desvantagens:
 - adiciona objetos extras;
 - pode ser desnecessário em coleções simples;
 - às vezes a linguagem já fornece suporte nativo.
- Relação com princípios de projeto: melhora encapsulamento e baixo acoplamento com a estrutura de dados.
- Quando costuma aparecer: quando o cliente precisa percorrer coleções de forma uniforme.

Mediator

- Categoria: Comportamental
- Problema que soluciona: comunicação complexa entre muitos objetos que se conhecem demais.
- Definição: centraliza a comunicação entre objetos em um mediador.
- Como funciona: objetos conversam com o mediador, e o mediador coordena as interações.
- Vantagens:
 - reduz acoplamento;
 - centraliza regras de interação;
 - melhora organização;
 - separa responsabilidades.
- Desvantagens:
 - o mediador pode virar um "objeto deus";
 - adiciona indireção;
 - pode concentrar complexidade.
- Relação com princípios de projeto: favorece baixo acoplamento e separação de responsabilidades, mas exige cuidado com coesão.
- Quando costuma aparecer: quando vários objetos precisam colaborar sem se referenciar diretamente o tempo todo.

Memento

- Categoria: Comportamental
- Problema que soluciona: salvar e restaurar estado sem expor detalhes internos.

- Definição: captura o estado interno de um objeto e permite restaurá-lo depois.
- Como funciona: o originador cria um memento; um cuidador guarda o memento; depois o estado pode ser restaurado.
- Vantagens:
 - preserva encapsulamento;
 - facilita desfazer;
 - separa responsabilidades;
 - organiza histórico de estado.
- Desvantagens:
 - pode consumir memória;
 - pode gerar muitos objetos;
 - controlar versões de estado pode ser difícil.
- Relação com princípios de projeto: ajuda encapsulamento e separação de responsabilidades.
- Quando costuma aparecer: quando é preciso desfazer ações ou restaurar versões anteriores de um objeto.

Observer

- Categoria: Comportamental
- Problema que soluciona: atualização automática de vários objetos quando um estado muda.
- Definição: define uma relação um-para-muitos entre sujeito e observadores.
- Como funciona: o sujeito mantém uma lista de observadores e notifica todos quando muda.
- Vantagens:
 - reduz acoplamento;
 - favorece extensão;
 - separa responsabilidades;
 - facilita reuso de observadores;
 - programa para interfaces.
- Desvantagens:
 - pode ser difícil rastrear notificações;
 - pode gerar cascata de atualizações;
 - pode complicar depuração.
- Relação com princípios de projeto: ajuda Aberto/Fechado, Inversão de Dependência e baixo acoplamento.
- Quando costuma aparecer: quando várias partes precisam reagir a mudanças de um estado central.

State

- Categoria: Comportamental
- Problema que soluciona: muitos condicionais ligados ao estado interno.
- Definição: permite que um objeto altere seu comportamento quando seu estado muda.
- Como funciona: cada estado vira uma classe; o contexto delega o comportamento ao estado atual.
- Vantagens:
 - reduz if/switch;
 - encapsula variação;
 - melhora extensão;
 - separa responsabilidades.

- Desvantagens:
 - aumenta número de classes;
 - pode parecer parecido com Strategy;
 - pode ser exagero para poucos estados.
- Relação com princípios de projeto: favorece Aberto/Fechado, coesão e baixo acoplamento.
- Quando costuma aparecer: quando o comportamento de um objeto depende claramente de seu estado interno.

Strategy

- Categoria: Comportamental
- Problema que soluciona: variação de algoritmo sem alterar a classe que o usa.
- Definição: encapsula algoritmos intercambiáveis em classes próprias.
- Como funciona: o contexto recebe uma estratégia e delega a ela o comportamento variável.
- Vantagens:
 - reduz acoplamento;
 - encapsula variação;
 - favorece extensão;
 - evita condicionais grandes;
 - melhora manutenibilidade.
- Desvantagens:
 - cria mais classes;
 - pode ser complexidade desnecessária para poucos comportamentos;
 - exige escolher a estratégia certa.
- Relação com princípios de projeto: ajuda Aberto/Fechado, Inversão de Dependência e separação de responsabilidades.
- Quando costuma aparecer: quando há vários algoritmos possíveis e o cliente precisa trocar o comportamento.

Template Method

- Categoria: Comportamental
- Problema que soluciona: código repetido em algoritmos com variações pequenas.
- Definição: define o esqueleto de um algoritmo em uma classe base e deixa passos para subclasses.
- Como funciona: o método principal fica na superclasse; as subclasses implementam ou personalizam etapas.
- Vantagens:
 - evita duplicação;
 - organiza fluxo fixo;
 - separa partes variáveis;
 - favorece reuso por herança.
- Desvantagens:
 - depende de herança;
 - pode tornar o projeto rígido;
 - pode dificultar alteração posterior.
- Relação com princípios de projeto: ajuda Inversão de Controle, Aberto/Fechado e separação de responsabilidades.

- Quando costuma aparecer: quando várias classes seguem o mesmo processo geral, mas com passos específicos diferentes.

Visitor

- Categoria: Comportamental
- Problema que soluciona: novas operações sobre uma hierarquia estável de classes.
- Definição: separa operações dos objetos visitados, colocando a lógica de operação em visitantes.
- Como funciona: cada elemento aceita um visitor e chama o método específico correspondente.
- Vantagens:
 - facilita novas operações;
 - separa responsabilidades;
 - favorece extensão de comportamento;
 - pode organizar múltiplas operações sobre a mesma hierarquia.
- Desvantagens:
 - pode quebrar encapsulamento;
 - se surgir novo tipo de elemento, vários visitors mudam;
 - aumenta complexidade e número de classes.
- Relação com princípios de projeto: favorece Aberto/Fechado em relação a novas operações, mas pode afetar encapsulamento.
- Quando costuma aparecer: quando a hierarquia de objetos muda pouco, mas as operações mudam bastante.

Como diferenciar padrões parecidos

- Factory Method x Abstract Factory
 - Factory Method decide uma criação por vez.
 - Abstract Factory cria famílias de objetos relacionados.
- Strategy x State
 - Strategy troca algoritmos escolhidos pelo cliente.
 - State troca comportamento conforme o estado interno.
- Decorator x Proxy
 - Decorator adiciona responsabilidade.
 - Proxy controla acesso ou representação do objeto.
- Adapter x Facade
 - Adapter converte uma interface em outra.
 - Facade simplifica o uso de um subsistema.
- Composite x Decorator
 - Composite representa árvore de parte e todo.
 - Decorator empacota objeto para adicionar comportamento.
- Command x Chain of Responsibility

- Command encapsula uma ação em um objeto.
- Chain of Responsibility passa a requisição até alguém tratar.
- Memento x Prototype
 - Memento guarda estado para restaurar depois.
 - Prototype cria novo objeto por cópia de outro.
- Observer x Mediator
 - Observer notifica vários interessados sobre mudança.
 - Mediator centraliza a comunicação entre colegas.